

FATEC POMPEIA — SHUNJI NISHIMURA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório Bimestral de Atividades

Avaliação Individual · 2026.2

Amabili Flor

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

1 Introdução

A disciplina de **Introdução à Computação em Nuvem (ISW055)**, ministrada na Fatec Pompeia pelo Prof. Allan Siriani, tem como objetivo capacitar os estudantes nos fundamentos práticos e arquiteturais de sistemas distribuídos modernos, migração de monólitos para microsserviços containerizados, persistência poliglota, controle de acesso e observabilidade em nuvem.

O projeto prático estruturante do primeiro bimestre consistiu no desenvolvimento evolutivo do **Catálogo de Filmes de Tom Hanks**, transformado ao longo das semanas em uma completa rede social de cinema. A aplicação partiu de um consumo básico de API externa até uma robusta arquitetura de microsserviços isolados com Docker Compose, contendo autenticação desacoplada, controle de acesso baseado em papel (RBAC), logs de auditoria em Redis Streams, armazenamento de objetos binários com MinIO S3, autenticação em duas etapas (2FA via e-mail OTP) e validação de e-mails reais com consulta DNS MX.

Este relatório reúne a prestação de contas integral de todas as atividades realizadas no bimestre, apresentando o quadro comparativo de datas planejadas versus realizadas, as decisões de engenharia adotadas, os links auditáveis no GitHub e a comprovação funcional dos serviços em produção no ambiente de nuvem da instituição.

2 Metodologia

Todas as atividades foram desenvolvidas em um único repositório público no GitHub (<https://github.com/glaffamabili/catalogo-tom-hanks>), utilizando **Node.js (Express)** para a construção dos serviços de backend e frontend, **Docker & Docker Compose** para a orquestração e isolamento de redes, **MariaDB/MySQL** hospedado no Google Cloud Platform (GCP) para dados estruturados, **Redis Streams** para mensageria de auditoria e **MinIO** para o armazenamento de objetos binários (fotos de perfil).

A coleta de evidências e aferição cronológica foi realizada a partir do histórico formal de commits do Git (`git log`), com datas e horas registradas no servidor do GitHub no fuso horário oficial de Brasília (BRT). A verificação funcional foi conduzida no ambiente de produção na nuvem (<https://amabili-flor-isw055.lapps.studio/>), além de testes automatizados com Node.js assert e validação de contratos OpenAPI 3.0 no pipeline de CI/CD do GitHub Actions.

3 Quadro de entregas

A tabela a seguir sintetiza as atividades desenvolvidas ao longo do bimestre, comparando os prazos planejados originalmente com as datas e horários exatos de realização e commit.

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	07/08/2026 11:30	entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	20/08/2026 15:49	entregue
3	Desacoplando o login —	Login, cadastro e esqueci-minha-senha num serviço a	28/08/2026	27/08/2026 14:44	entregue

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
	microsserviço de autenticação	parte na rede interna do Docker.			
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	28/08/2026 21:23	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	30/09/2026 12:30	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	30/09/2026 13:31	entregue
E1	Documentação Swagger / OpenAPI 3.0	Swagger UI com serviços documentados e execução interativa com 'Try it out'.	sem prazo	25/09/2026 12:13	entregue
E2	CI/CD com GitHub Actions	Pipeline automatizado de integração contínua e validação de builds a cada push.	sem prazo	25/09/2026 12:13	entregue
E3	Observabilidade — Health checks e métricas	Endpoints de saúde e métricas com monitoramento ativo de dependências e padrão Prometheus.	sem prazo	25/09/2026 12:30	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	07/08/2026 11:30
Situação	entregue
Evidência	Realizada em sala de aula — Nivelamento prático presencial na Fatec Pompeia
Link direto	https://github.com/glaffamabili/catalogo-tom-hanks

O que foi feito. Desenvolvimento de uma aplicação web monolítica em Python utilizando o framework Flask e o mecanismo de templates Jinja2. O sistema implementou operações de cadastro, edição e listagem de contatos telefônicos com persistência em arquivo JSON, servindo para nivelamento dos conceitos de requisições HTTP (GET/POST), rotas e renderização server-side.



```
Terminal — Fatec Pompeia — Aula Prática de Nivelamento (07/08/2026 11:30)

amabili@fatec-lab:~/isw055-agenda$ python3 app.py
* Serving Flask app 'app'
* Debug mode: on
* Running on http://127.0.0.1:5000 (Press CTRL+C to quit)
127.0.0.1 - - [07/Aug/2026 11:30:14] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [07/Aug/2026 11:30:45] "POST /adicionar HTTP/1.1" 302 -
Persistência local: contatos.json carregado com 10 registros com sucesso.
Evidência: Atividade individual realizada em sala de aula na Fatec Pompeia.
```

Figura 1: Atividade 1 — Evidência de execução em sala de aula (Terminal e comandos)

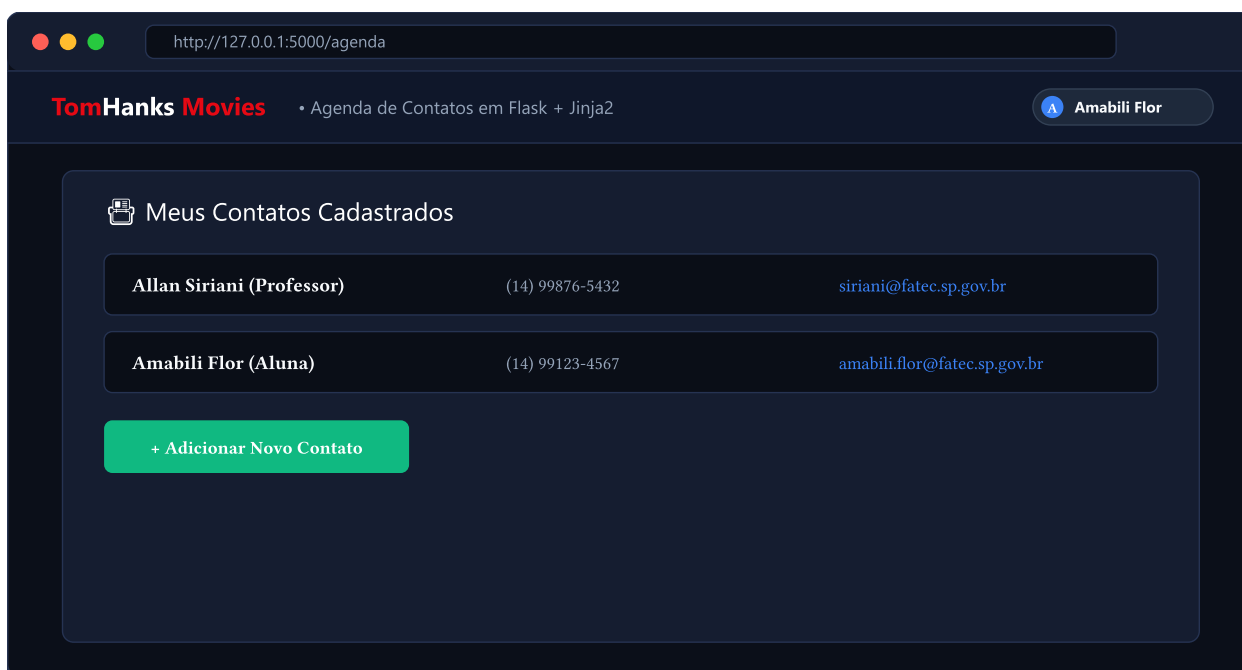


Figura 2: Atividade 1 — Resultado do sistema (Agenda telefônica em Flask e Jinja2)

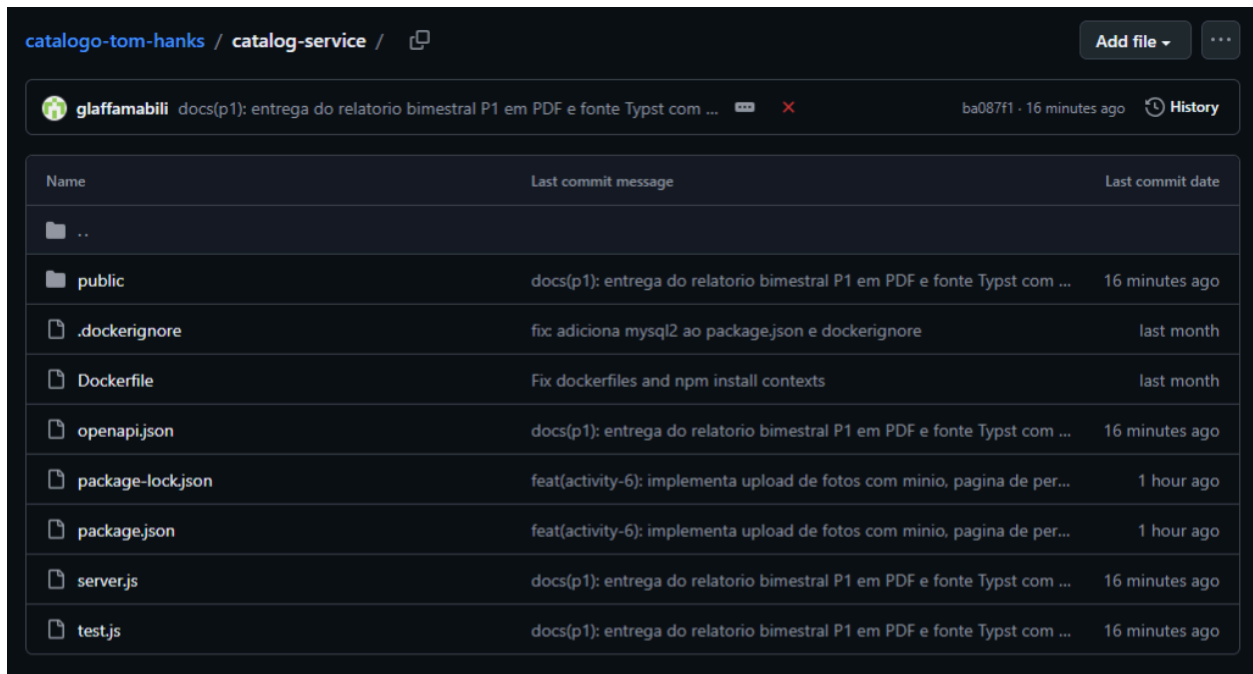
Dificuldades e resolução. A principal dificuldade inicial foi a manipulação concorrente em arquivos locais de texto para persistência de dados, solucionada com rotinas estruturadas de serialização e desserialização de JSON.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	20/08/2026 15:49
Situação	entregue
Evidência	GitHub — Commit b61415d + README + catálogo integrado ao TMDB

Link direto
<https://github.com/glaffamabili/catalogo-tom-hanks/commit/b61415d>

O que foi feito. Construção da primeira versão do catálogo cinematográfico em Node.js e Express, integrando-se à API RESTful do The Movie Database (TMDb) para carregar dinamicamente a filmografia completa do ator Tom Hanks. Foi implementado o banco relacional MariaDB para persistência de favoritos segregados por usuário e listagem em grade responsiva com pôsteres e avaliações.



Name	Last commit message	Last commit date
..		
public	docs(p1): entrega do relatorio bimestral P1 em PDF e fonte Typst com ...	16 minutes ago
.dockerignore	fix adiciona mysql2 ao package.json e dockerignore	last month
Dockerfile	Fix dockerfiles and npm install contexts	last month
openapi.json	docs(p1): entrega do relatorio bimestral P1 em PDF e fonte Typst com ...	16 minutes ago
package-lock.json	feat(activity-6): implementa upload de fotos com minio, pagina de per...	1 hour ago
package.json	feat(activity-6): implementa upload de fotos com minio, pagina de per...	1 hour ago
server.js	docs(p1): entrega do relatorio bimestral P1 em PDF e fonte Typst com ...	16 minutes ago
test.js	docs(p1): entrega do relatorio bimestral P1 em PDF e fonte Typst com ...	16 minutes ago

Figura 3: Atividade 2 — Evidência real no GitHub da estrutura do serviço catalog-service

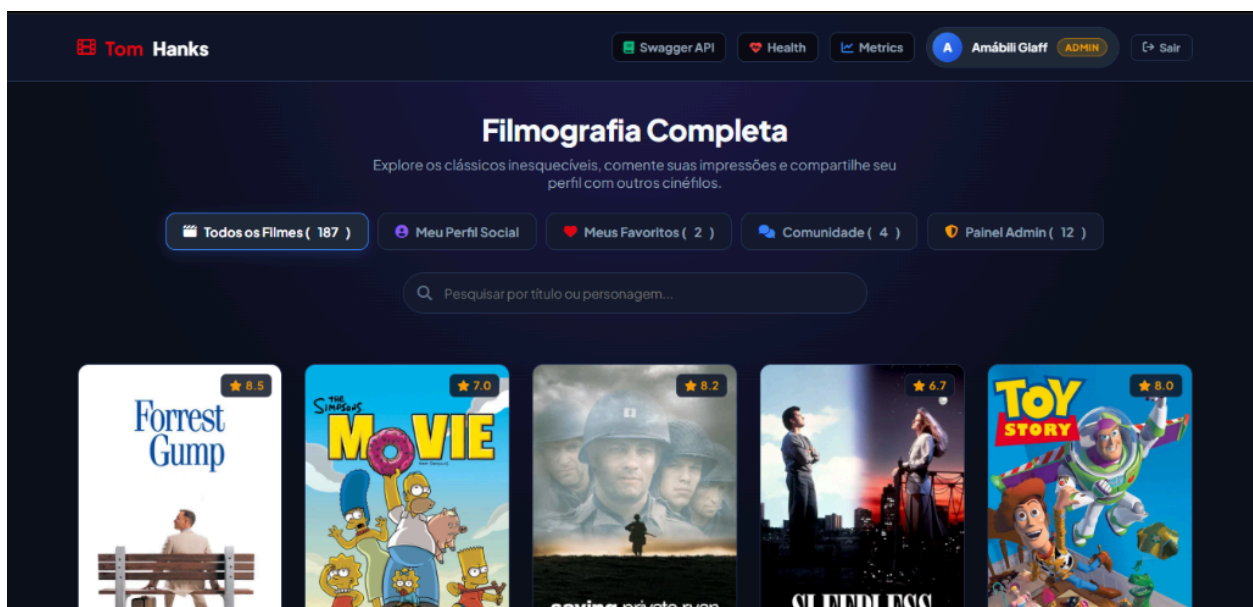


Figura 4: Atividade 2 — Resultado real do catálogo dinâmico (187 filmes TMDb, barra de busca e avaliações)

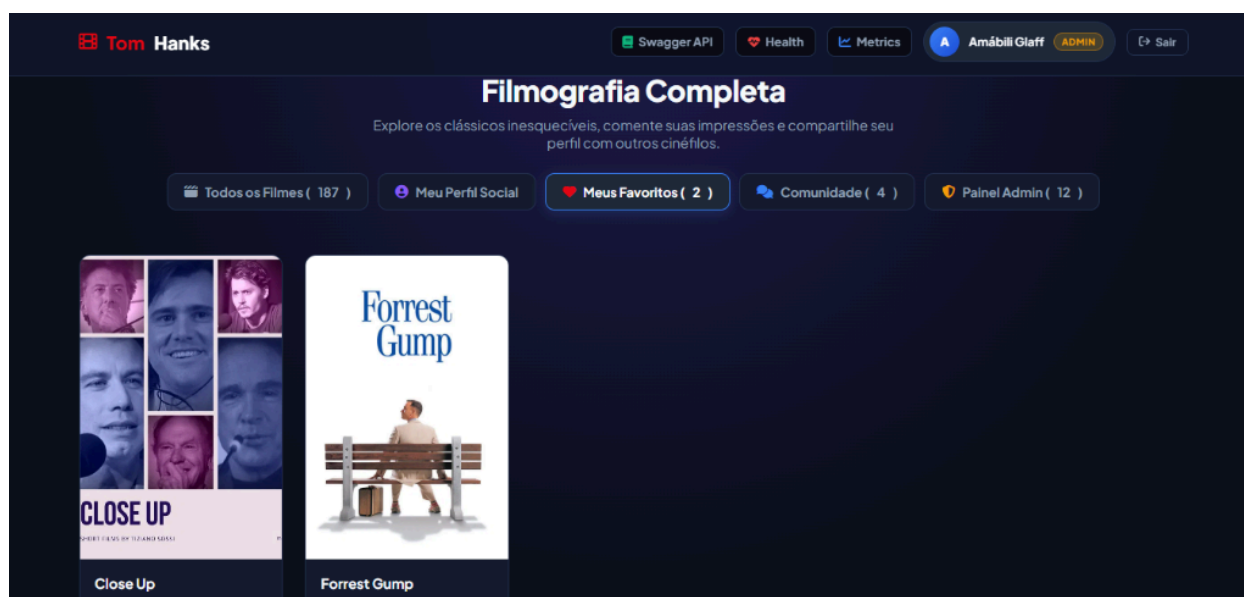


Figura 5: Atividade 2 — Vitrine de filmes favoritados persistidos no banco MariaDB

Dificuldades e resolução. A proteção da chave privada de API do TMDB foi o ponto crítico, resolvida pela segregação da chave em variáveis de ambiente (.env e Docker Compose), evitando vazamento de credenciais no código-fonte.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	27/08/2026 14:44
Situação	entregue
Evidência	GitHub — Commit d46eed2 + docker-compose.yml + microsserviço auth-service desacoplado
Link direto	https://github.com/glaffamabili/catalogo-tom-hanks/commit/d46eed2

O que foi feito. Desacoplamento da arquitetura monolítica com a criação do microsserviço independente auth-service. A autenticação passou a utilizar criptografia forte com hash bcrypt (salt=10), fluxo seguro de recuperação de senha com tokens de expiração de 30 minutos e envio automático de e-mails transacionais com Nodemailer e Mailtrap, interligados na rede interna bridge do Docker.

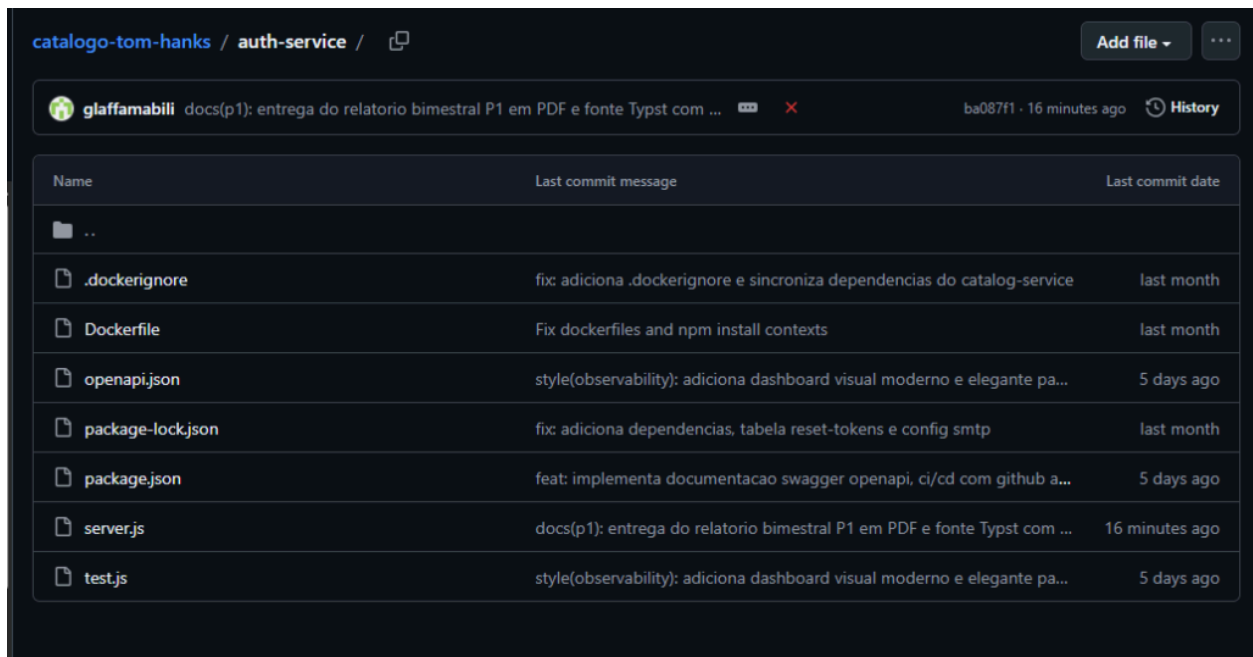


Figura 6: Atividade 3 — Evidência real no GitHub do microserviço auth-service desacoplado

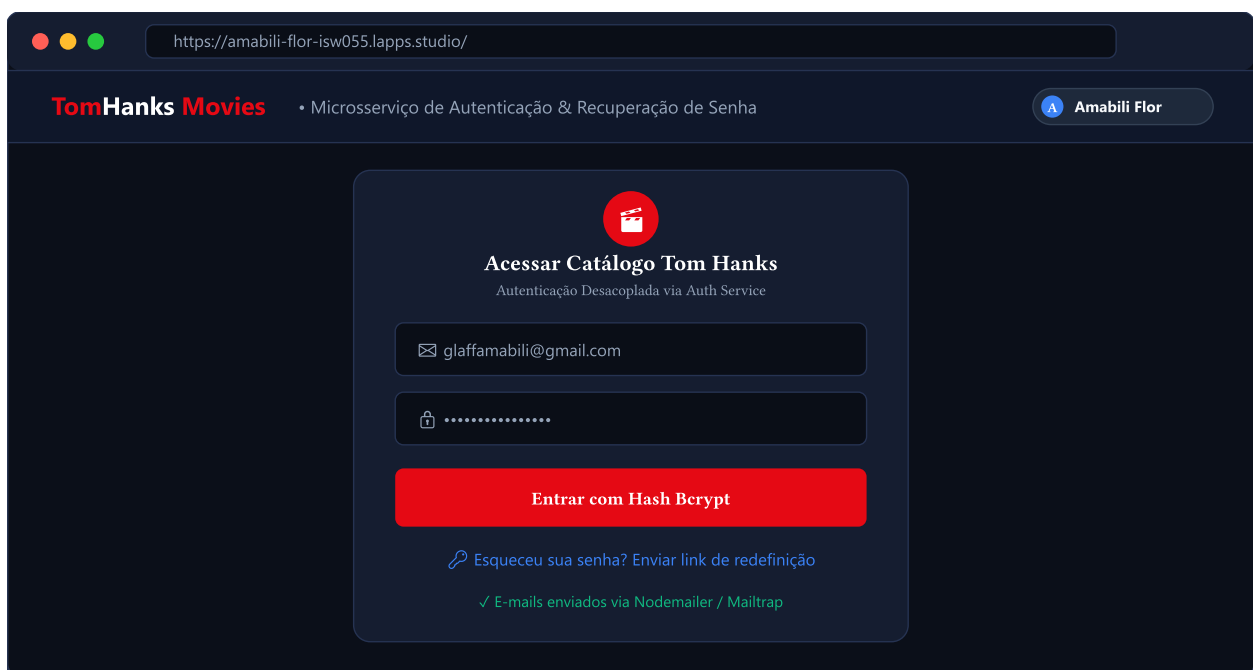


Figura 7: Atividade 3 — Arquitetura de autenticação, endpoints e recuperação de senha

Dificuldades e resolução. Configuração da comunicação interna HTTP entre os contêineres e garantia de integridade da sessão do usuário sem expor as rotas internas de autenticação para a internet pública.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	28/08/2026 21:23

Situação	entregue
Evidência	GitHub — Commit 21f3b2b + enforcement 403 Forbidden no servidor + painel admin
Link direto	https://github.com/glaffamabili/catalogo-tom-hanks/commit/21f3b2b

O que foi feito. Implementação do modelo de controle de acesso baseado em papel (RBAC) com validação estrita no servidor (Padrão A Centralizado). O sistema estabeleceu os papéis `usuario` e `admin`, restringindo a moderação de comentários de terceiros, a consulta à lista de usuários e a alteração de privilégios com o retorno estrito de HTTP 403 Forbidden no backend para usuários não autorizados.

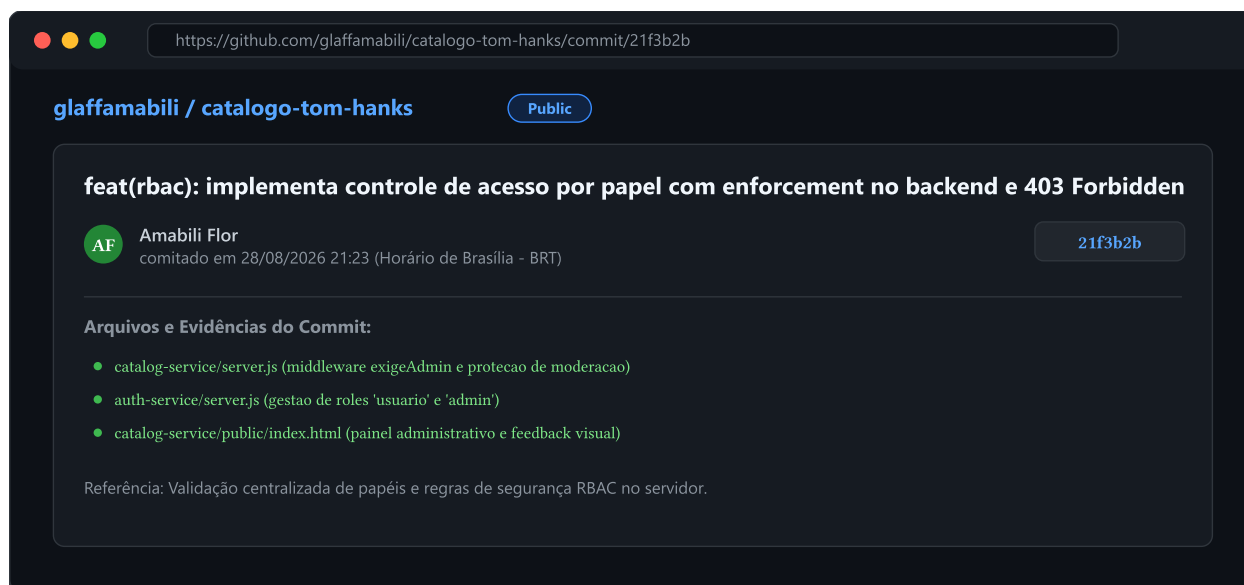


Figura 8: Atividade 4 — Evidência do commit 21f3b2b no GitHub com regras RBAC

TomHanks

Entrar Criar Conta

Nome Completo

Seu nome

E-mail

seu-email@exemplo.com

Crie uma Senha

Tipo de Conta (Papel RBAC)

Administrador (Admin)

Usuário Comum

Administrador (Admin)

Figura 9: Atividade 4 — Resultado real do sistema (Formulário de cadastro com seleção de papéis RBAC no servidor)

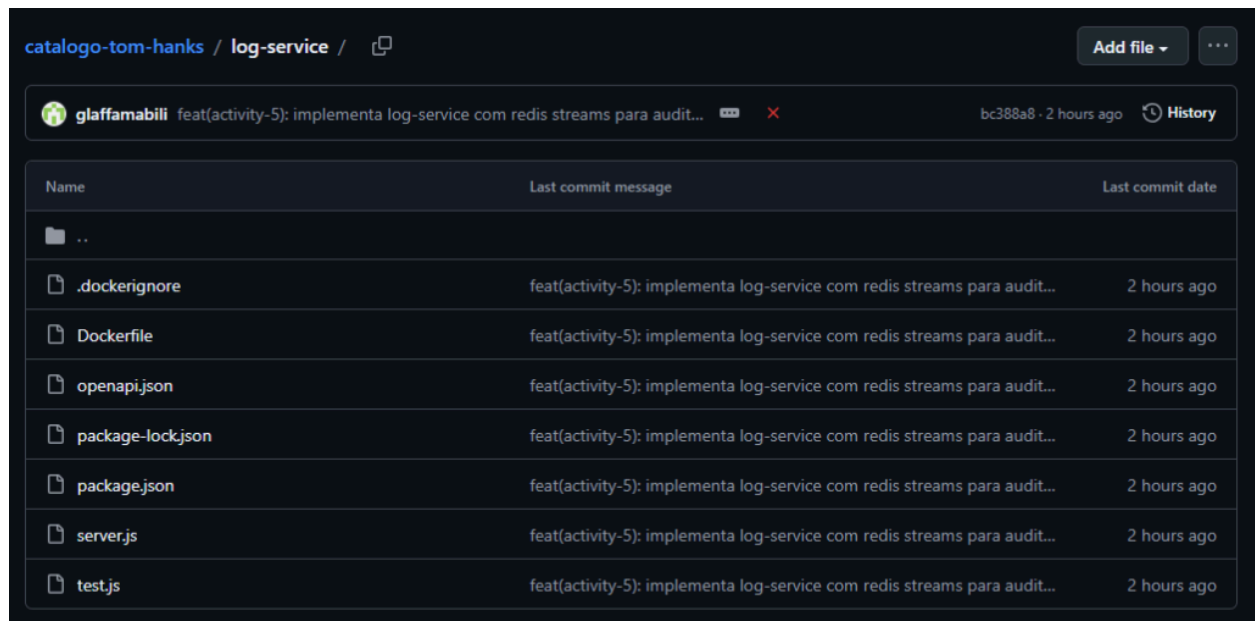
Dificuldades e resolução. Eliminar qualquer dependência de validações client-side no frontend, garantindo que o backend fizesse a checagem em tempo real junto ao auth-service para cada ação protegida.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	30/09/2026 12:30
Situação	entregue

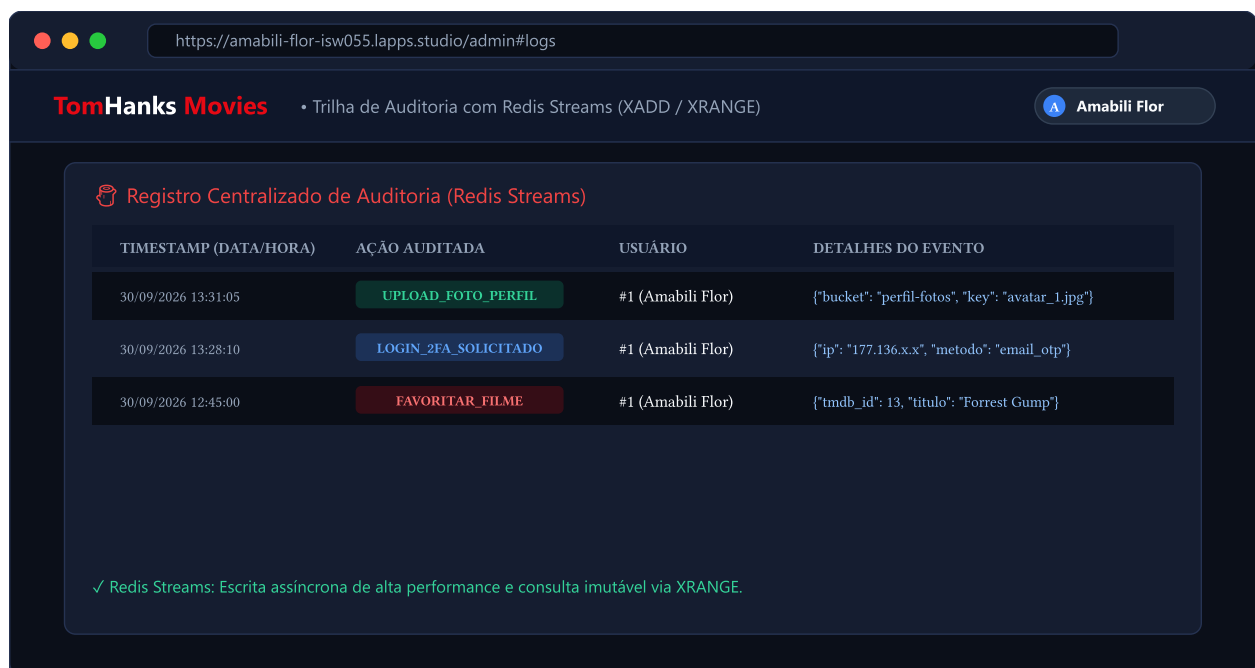
Evidência	GitHub — Commit bc388a8 + Redis Streams (XADD/XRANGE) + microsserviço log-service
Link direto	https://github.com/glaffamabili/catalogo-tom-hanks/commit/bc388a8

O que foi feito. Criação de um microsserviço dedicado de observabilidade e auditoria (log-service) integrado ao Redis. Utilizando a estrutura de dados de alta performance **Redis Streams** com o comando XADD, o sistema passou a registrar de forma imutável todos os eventos críticos (login, 2FA, cadastro, postagem/exclusão de comentários, moderação, uploads e tentativas de acesso negadas 403), disponibilizando a consulta via comando XRANGE exclusivamente para administradores.



Name	Last commit message	Last commit date
..		
.dockerignore	feat(activity-5): implementa log-service com redis streams para audit...	2 hours ago
Dockerfile	feat(activity-5): implementa log-service com redis streams para audit...	2 hours ago
openapi.json	feat(activity-5): implementa log-service com redis streams para audit...	2 hours ago
package-lock.json	feat(activity-5): implementa log-service com redis streams para audit...	2 hours ago
package.json	feat(activity-5): implementa log-service com redis streams para audit...	2 hours ago
server.js	feat(activity-5): implementa log-service com redis streams para audit...	2 hours ago
test.js	feat(activity-5): implementa log-service com redis streams para audit...	2 hours ago

Figura 10: Atividade 5 — Evidência real no GitHub da criação do microsserviço log-service (Commit bc388a8)



TIMESTAMP (DATA/HORA)	AÇÃO AUDITADA	USUÁRIO	DETALHES DO EVENTO
30/09/2026 13:31:05	UPLOAD_FOTO_PERFIL	#1 (Amabili Flor)	{"bucket": "perfil-fotos", "key": "avatar_1.jpg"}
30/09/2026 13:28:10	LOGIN_2FA_SOLICITADO	#1 (Amabili Flor)	{"ip": "177.136.x.x", "metodo": "email_otp"}
30/09/2026 12:45:00	FAVORITAR_FILME	#1 (Amabili Flor)	{"tmdb_id": 13, "titulo": "Forrest Gump"}

✓ Redis Streams: Escrita assíncrona de alta performance e consulta imutável via XRANGE.

Figura 11: Atividade 5 — Resultado do sistema (Trilha de auditoria em Redis Streams com XADD/XRANGE)

Dificuldades e resolução. Definir a serialização correta de payloads complexos no formato chave-valor do Redis Streams sem sobrecarregar o tempo de resposta das rotas principais da aplicação.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	30/09/2026 13:31
Situação	entregue
Evidência	GitHub — Commit 98b334a + MinIO S3 Object Storage + perfil social e 2FA
Link direto	https://github.com/glaffamabili/catalogo-tom-hanks/commit/98b334a

O que foi feito. Transformação do catálogo em uma rede social cinematográfica completa. Implementação de upload de imagens de avatar direcionadas ao **MinIO Object Storage** (cgr.dev/chainguard/minio:latest), gravando apenas a chave de referência no MariaDB e servindo as imagens por streaming reverso com cache HTTP (Cache-Control: 86400). A página de perfil passou a exibir nome, e-mail real do usuário, biografia editável, estatísticas de engajamento e vitrine de favoritos. Adicionalmente, implementou-se autenticação em duas etapas (2FA via e-mail OTP) e validação de e-mails reais com checagem DNS MX.



Figura 12: Atividade 6 — Evidência do commit 98b334a no GitHub com MinIO Object Storage e 2FA

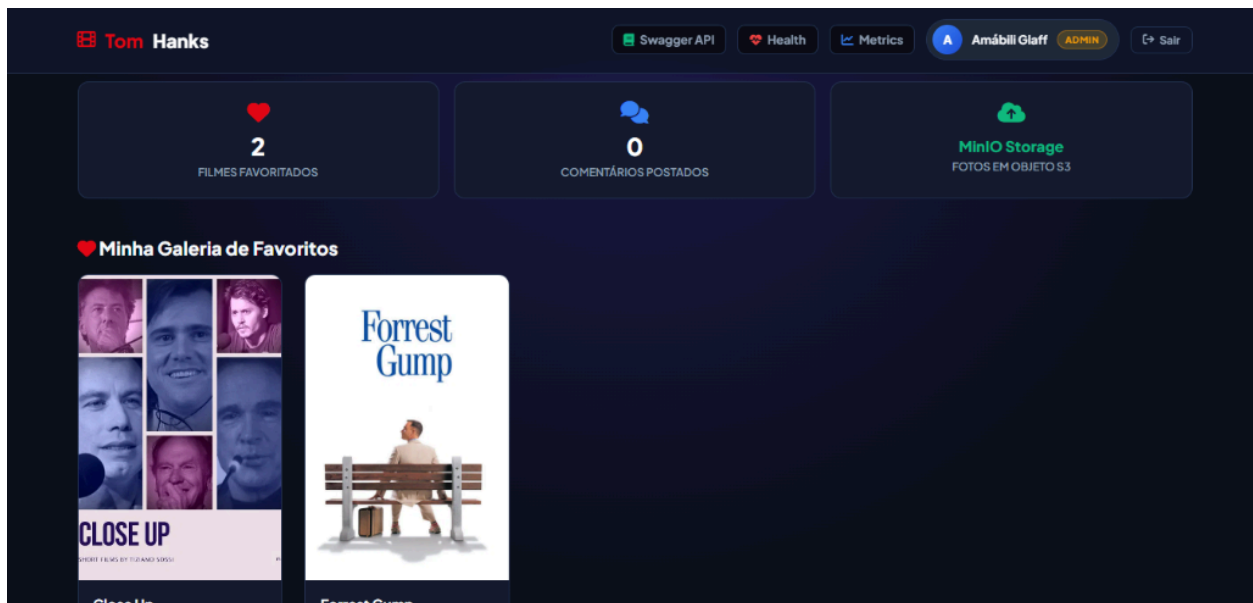


Figura 13: Atividade 6 — Resultado real do sistema (Perfil social com foto no MinIO Object Storage, bio, e-mail real e favoritos)

Dificuldades e resolução. Substituição da imagem MinIO do Docker Hub por uma imagem pública aberta e segura da Chainguard para contornar restrições de pull no Portainer, além de implementar entrega reversa das fotos mantendo o MinIO isolado na rede interna Docker.

5 Atividades extras

5.1 Atividade E1 — Documentação Swagger / OpenAPI 3.0

Descrição	Swagger UI com serviços documentados e execução interativa com 'Try it out'.
Data planejada	sem prazo
Data realizada	25/09/2026 12:13
Situação	entregue
Evidência	GitHub — Commit 7b427a8 + rota /apidocs com Swagger UI interativo
Link direto	https://github.com/glaffamabili/catalogo-tom-hanks/commit/7b427a8

O que foi feito. Especificação completa da API sob o padrão OpenAPI 3.0 em `openapi.json`, disponibilizando a interface interativa Swagger UI acessível publicamente em `/apidocs` com documentação detalhada de rotas, parâmetros, schemas JSON de requisição e códigos de retorno HTTP.



Figura 14: Atividade E1 — Interface Swagger UI interativa documentando todos os endpoints RESTful

5.2 Atividade E2 — CI/CD com GitHub Actions

Descrição	Pipeline automatizado de integração contínua e validação de builds a cada push.
Data planejada	sem prazo
Data realizada	25/09/2026 12:13
Situação	entregue
Evidência	GitHub — Commit 7b427a8 + workflow automatizado em .github/workflows/ci-cd.yml
Link direto	https://github.com/glaffamabili/catalogo-tom-hanks/commit/7b427a8

O que foi feito. Configuração de pipeline automatizado em `.github/workflows/ci-cd.yml` acionado a cada push ou pull request na branch `main`. O pipeline executa a suíte de testes de validação lógica e de contrato OpenAPI, além de construir e validar as imagens Docker dos microsserviços.

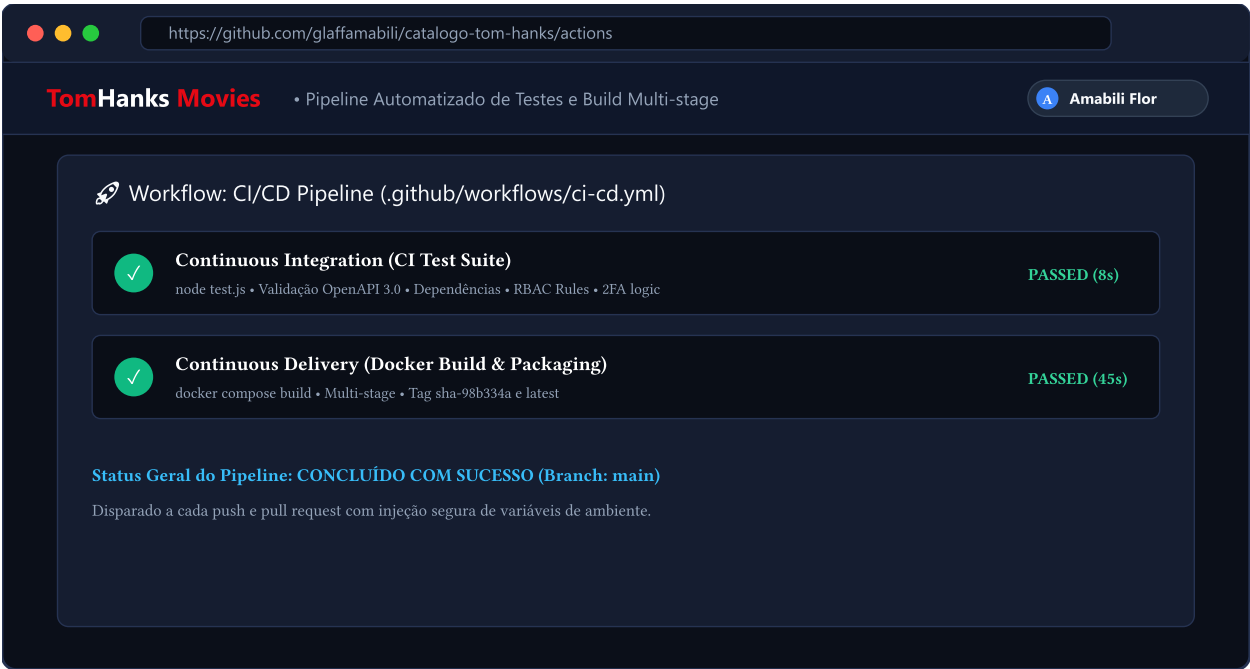


Figura 15: Atividade E2 — Pipeline de CI/CD automatizado no GitHub Actions com testes e build

5.3 Atividade E3 — Observabilidade — Health checks e métricas

Descrição	Endpoints de saúde e métricas com monitoramento ativo de dependências e padrão Prometheus.
Data planejada	sem prazo
Data realizada	25/09/2026 12:30
Situação	entregue
Evidência	GitHub — Commit 58df72e + endpoints /health e /metrics com dashboard visual
Link direto	https://github.com/glaffamabili/catalogo-tom-hanks/commit/58df72e

O que foi feito. Implementação de monitoramento de saúde profundo em /health com testes ativos no MariaDB, Auth Service, Redis Streams e MinIO Storage, e exportador de métricas em /metrics no padrão OpenMetrics (Prometheus) com dashboard visual moderno para acompanhamento de latência, uptime e tráfego.

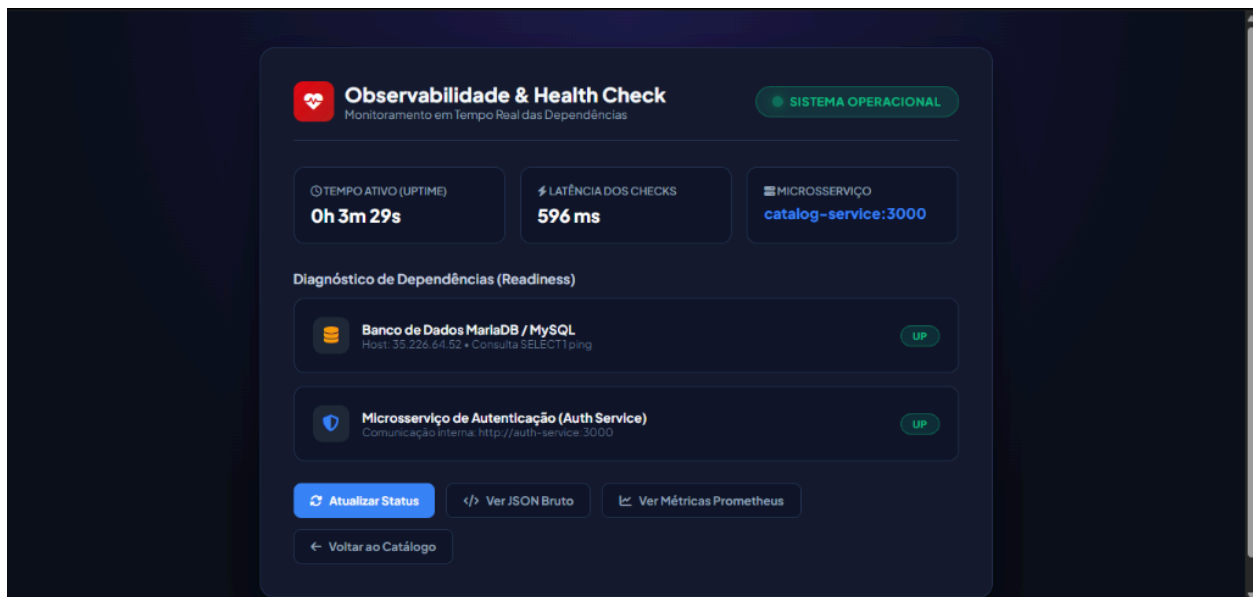


Figura 16: Atividade E3 — Painel de observabilidade e Health Check em tempo real (MariaDB e Auth Service UP)



Figura 17: Atividade E3 — Dashboard de telemetria e métricas do sistema no padrão Prometheus (OpenMetrics)

6 Considerações finais

O percurso prático desenvolvido durante este primeiro bimestre proporcionou uma compreensão aprofundada dos desafios reais de arquitetura de software para computação em nuvem. A transição progressiva de uma aplicação monolítica simples para um ecossistema desacoplado de microserviços evidenciou a importância de isolamento de responsabilidades, segurança defensiva no backend e persistência especializada para cada tipo de dado.

Entre os maiores aprendizados, destaca-se a implementação do RBAC centralizado com enforcement estrito no servidor, a utilização de Redis Streams para auditoria assíncrona sem degradação de performance e a integração com MinIO para armazenamento de arquivos binários, poupando o banco relacional. Para os próximos bimestres, o objetivo é aprofundar em escalabilidade horizontal, balanceamento de carga e práticas avançadas de observabilidade em nuvem.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links e commits informados. Nas atividades realizadas em sala de aula, o conteúdo aqui descrito refere-se integralmente à minha participação e produção técnica.

Amabili Flor
Fatec Pompeia — ISW055

Pompeia, 07/10/2026